

HiDiTingV100 常见问题

FAQ

文档版本	00B01
发布日期	2025-06-05

前言

概述

本文档主要介绍 HiDiTing 解决方案中常见的问题处理和解决办法。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
 危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危

符号	说明
	害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....	i
1 媒体常见问题 FAQ	1
1.1 其他视频格式转换成当前支持的视频格式问题.....	1
1.2 媒体 sample 限制说明.....	3
1.3 媒体常见问题定位解答.....	5
2 JS 应用常见问题 FAQ.....	7
2.1 canvas 基本用法.....	7
2.2 动画设置 background-image 设置.....	8
2.3 css 样式中设置样式可以通过动态绑定.....	8
2.4 组件设置背景图.....	9
2.5 JS 文件超过 30KB 报错 “xxx.js is bigger than 30 KB.”	11
2.6 player 设置 streamType	11
3 图形常见问题 FAQ	12
3.1 图形显示问题及建议.....	12
3.1.1 花屏问题.....	12
3.1.2 黑屏问题.....	13
3.1.3 冻屏问题.....	14
3.1.4 一级跳转建议	14
3.1.5 控件内容未显示.....	14
3.1.6 按钮图标太小，不易点击	14
3.1.7 支付宝认证：将无法显示的文字替换为特定的字符	15
3.1.8 UIList 滑动到页面底部时，进度条显示不到 100%.....	15
3.1.9 控件析构时报错：parent is not nullptr	16

3.1.10 UIChartPolyLine 如何合理呈现心率数据	16
3.1.11 UIScrollView 使用建议	16
3.1.12 事件分发和响应流程.....	16
3.1.12.1 表冠事件.....	16
3.1.12.2 按键事件.....	17
3.1.12.3 触屏类事件	17
3.1.13 自适应刷新频率.....	18
3.1.14 功能宏配置约束.....	18
3.1.15 多行文字换行时，截断单词.....	18
3.2 视频显示问题	19
3.2.1 视频播放状态异常	19
3.2.1.1 媒体线程创建失败	19
3.2.1.2 内存不足，申请不到视频帧.....	19
3.2.1.3 视频状态错误	19
3.3 内存问题.....	20
3.3.1 如何快速定位页面内部内存泄漏	20
3.3.2 如何使用 liteos 定位踩内存	22
3.3.3 运行期间越来越卡	25
3.3.4 一级卡片左右滑动时，卡片内容呈现方形而非圆形	25
3.3.5 多线程操作 UI	26
4 BTH 特性使用建议.....	28
4.1 对外接口回调函数使用约束.....	28
4.2 数传传输场景使用建议	28
4.3 蓝牙连接相关	29
4.4 BLE GATT 相关.....	30
4.5 协议栈开关	31
4.6 通话相关使用建议	31
5 音频常见问题 FAQ	34
5.1 音频问题定位方法	34
5.2 无声问题.....	35
5.3 卡顿问题.....	36

5.4 音量问题..... 36

Draft

插图目录

图 2-1 效果图 10

表格目录

表 5-1 各场景的音频通路及最低主频要求..... 34

Draft

1

媒体常见问题 FAQ

1.1 其他视频格式转换成当前支持的视频格式问题

1.2 媒体 sample 限制说明

1.3 媒体常见问题定位解答

1.1 其他视频格式转换成当前支持的视频格式问题

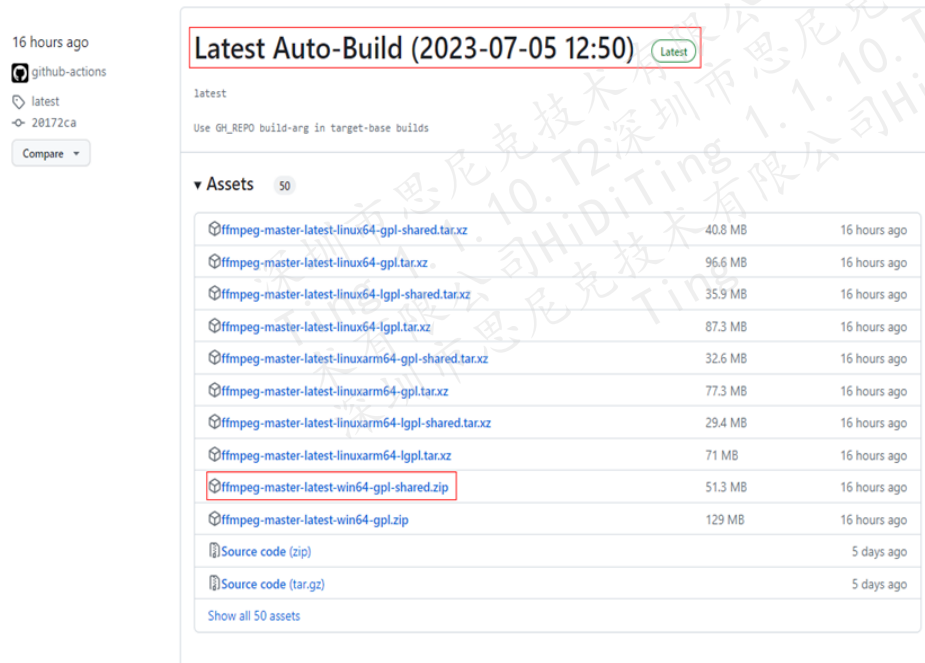
须知

- 目前只支持 MP4 封装格式的视频文件。
- 目前只支持 MJPEG 格式的视频流。

步骤 1 准备 FFMPEG 工具。

下载网址: <https://github.com/BtbN/FFmpeg-Builds/releases>

直接取最新版本即可, 下载压缩包, 解压到本地, 如下图所示:



步骤 2 使用 FFMPEG 程序转换视频。

将视频裁剪成方的（比例 1：1），原视频：1920×816，裁剪后：816×816；

将 mov 格式转成 mp4 格式，在“ffmpeg-master-latest-win64-gpl-shared\bin”目录，shift+鼠标右键打开 PowerShell 终端，在终端中输入 ffmpeg 命令。

ffmpeg 命令如下：.ffmpeg.exe -

```
i .\14_AVC_Main@L4.0_9517Kbps_1920x816_29.970fps_8bits_AAC_96.0Kbps_2ch  
.mov -vf crop=816:816 -threads 4 -preset ultrafast -strict -2 .\816x816.mp4
```

命令里面的参数命令都是 ffmpeg 的标准命令，可自行查询。

说明

裁剪会损失部分视频内容，请据实际视频内容划定 crop 区域（例子是居中裁剪）。

步骤 3 将视频缩放到 454×454，同时调整量化因子，保证画质。

ffmpeg 命令如下：.ffmpeg.exe -i .\816x816.mp4 -vf scale=454:454,setdar=1:1 -c:v
mjpeg -b:v 0 -q:v 9 -qmin 6 -qmax 15 454x454.mp4

说明

1. 量化因子根据实际画质调整，理论上量化因子参数越小画质越好。

2. 由于量化因子调整会涉及码率变化，量化因子越小码率越高。

综上：画面和码率需要客户调整，可能存在取舍。

步骤 4 去除视频的音频流：

ffmpeg 命令如下：.ffmpeg.exe -i input.mp4 -c copy -an output.mp4

其中:

- -i: 输入视频路径
- -c: copy 使用原视频的视频流编码格式
- -an: 去掉音频流

步骤 5 修改 mp4 文件里面的视频流的帧率:

ffmpeg 命令如下: .\ffmpeg.exe -i input_video.mp4 -r 25 output_video.mp4

其中:

- -i 表示输入视频路径
- -r 修改帧率

步骤 6 按照原始视频文件时长截取视频:

ffmpeg 命令如下: .\ffmpeg.exe -i .\454x454.mp4 -vcodec copy -acodec copy -ss 00:00 -to 00:57 .\454x454_57s.mp4

原始视频文件时长查看方法, 将文件直接拖到 “ffmpeg-master-latest-win64-gpl-shared\bin\ffplay.exe” 上面, 查看 “Duration: ” 字段。

```
Input #0, mov,mp4,m4a,3gp,3g2,mj2, from 'D:\Software\ffmpeg\2023-07-05\ffmpeg-master-latest-win64-gpl-shared\in\20230712-141409.mp4':
Metadata:
  major_brand      : mp42
  minor_version    : 1
  compatible_brands: isommp41mp42
  creation_time    : 2023-06-26T13:09:58.000000Z
  copyright        :
  copyright-eng    :
Duration: 00:00:56.57, start: 0.000000, bitrate: 1336 kb/s
Stream #0:0[0x1](und): Video: h264 (High) (avc1 / 0x31537661), yuv420p(tv, bt709, progressive), 720x1280, 1278 kb/s,
```

-----结束

1.2 媒体 sample 限制说明

- 音频播放 sample

路径:

middleware\services\media\foundation\sample\wrapper\player_sample_wrapper.cpp

对外接口:

- PlayerSample: 用于播放本地音乐, 包含本地蓝牙音乐。
- TonePlayerSampleTest: 用于播放本地系统提示音。

📖 说明

- PlayerSample 和 TonePlayerSampleTest 目前退出模式也是同步的，也就是传入 stop 指令以后会等待资源销毁完全再返回，同步就要求 start 指令和 stop 指令需要在同一个线程被调用，否则 stop 指令会被卡住。
- PlayerSample 和 TonePlayerSampleTest 可被同时调用，每个接口里面就会创建两个线程，需要注意资源、空间的消耗。
- 目前播放器支持的音频文件格式为：AAC、mp3、FLAC、OGG、WAV、SBC。

- 视频播放 sample

路径：

middleware\services\media\foundation\sample\wrapper\video_play_wrapper.cpp

对外接口：

- int32_t StartVideoPlay(void)：开始播放视频。
- int32_t StopVideoPlay(void)：停止播放视频。
- int32_t PauseVideoPlay(void)：暂停播放视频。
- int32_t ResumeVideoPlay(void)：继续播放视频。
- void SetVideoPlayLoop(bool isLoop)：设置视频循环播放，此接口需要在 StartVideoPlay 之前调用才会生效。
- void SetSyncExitMode(bool isSyncExitMode)：设置视频退出的模式。
- bool IsExitCompletely(void)：获取是否是完全退出的状态。

📖 说明

- 因资源的限制目前最多支持一路视频，所以 MediaVideoPlay 的对象，最好只有一个，必须要先 StopVideoPlay 退出视频播放，再 StartVideoPlay 开始播放。
- 默认是循环播放视频，可通过接口 SetVideoPlayLoop 修改，可在 StartVideoPlay 以后设置。
- 默认视频退出模式是同步的，也就是调用 StopVideoPlay 接口会等待所有的视频播放资源销毁再返回，这里有一个限制就是 StartVideoPlay 和 StopVideoPlay 必须在同一个线程被调用，否则会被卡住，如果使用接口 SetSyncExitMode 改成不同步的，则无此限制，但是需要保证时序，因为 StopVideoPlay 销毁资源是异步的会有一定的耗时。

- 音频录制 sample 说明

路径：

middleware\services\media\foundation\sample\wrapper\audio_capture_wrapper.cpp

对外接口：

int32_t AudioCaptureSample(int32_t argc, const char *argv[])：此接口是 AT 命令的入口，包含了所有的相关指令。

📖 说明

- 音频录制，目前只支持 mp3、pcm、silk 这个 3 种格式的录制。
- 音频录制过程中可以下发 stop 指令停止。
- 音频录制目前录制 1000 帧，完成后自动结束，暂时无法设置录制时长，可自行修改代码适配。
- 音频录制过程中原则上不允许再播放音频。

1.3 媒体常见问题定位解答

目前版本是没有打开媒体的日志，要定位媒体的问题最好先打开媒体日志。

打开方式：

步骤 1 找到日志文件：middleware\services\media\foundation\utils\src\media_log.cpp。

步骤 2 找到 “g_enabledLevel” 变量并修改其值为 MEDIA_LOG_INFO。

步骤 3 重新编译生成。

----结束

问题一：文件错误导致的异常

- 文件路径错误

在媒体日志中查找 “Realpath input file failed”，如果有则说明文件路径错误，需要排查文件路径是否正确，文件是否存在。

- IO 相关的报错：

一般是文件有问题导致，可以在其他播放器上面试一下，也有可能是文件系统出了问题，如果确认文件没问题，可以排查下文件系统。

- 需要做视频表盘的时候，播放带音频的视频文件导致的异常：

目前对纯视频播放做了限制，如果创建的是纯视频的 MediaVideoPlay 的对象，但是视频文件是带音频的就会出现报错，所以如果是做视频表盘一定要注意排查视频是否是带音频流的。

- 播放不支持的格式导致的错误：

需要提前去确认此格式的音视频是否都支持，不支持的格式也会有相应的报错。

问题二：场景的限制导致的异常

媒体这里所有的音频流之间都是有策略的。

举例：通话会打断所有的音乐，提示音和音乐可以混音。

不同的场景之间媒体配置了不同策略。

具体场景策略请参考《HiDiTingV100 多媒体软件 开发指南》的“音频中断策略管理”章节的详细说明。

Draft

JS 应用常见问题 FAQ

- 2.1 canvas 基本用法
- 2.2 动画设置 background-image 设置
- 2.3 css 样式中设置样式可以通过动态绑定
- 2.4 组件设置背景图
- 2.5 JS 文件超过 30KB 报错 "xxx.js is bigger than 30 KB."
- 2.6 player 设置 streamType

2.1 canvas 基本用法

检查 IDE 中是否包含 canvas 组件，在 IDE 中进入目录：“File” → “Settings” → “Appearance & Behavior” → “System Settings” → “HarmonyOS SDK”，获取 HarmonyOS SDK Location 目录，在 SDK 目录下进入 “js\2.1.1.21\build-tools\ace-loader\lib\templatel\lite-validator.js” 文件中，查看 litewearableTag 中是否包含 canvas 组件，没有 canvas 请添加（请参见《HiDiTingV100 OpenHarmony JS 应用开发 用户指南》的“扩展组件”章节）。

[illegible]

canvas 基本用法如下:

html:

```
<canvas class=" canvas" ref=" canvas_ref" />
```

css:

```
.canvas{  
    width:100%;  
    height:100%;  
    background-color:transparent;  
}
```

js:

```
let canvas = this.$refs.canvas_ref;  
let ctx = canvas.getContext( '2d' );  
ctx.beginPath();  
//绘制逻辑  
ctx.strokeStyle = '#857f61' //绘制画笔颜色  
ctx.moveTo(x1,y1);//移动画笔坐标  
ctx.lineTo(x2,y2);//绘制线条坐标  
ctx.stroke();
```

2.2 动画设置 background-image 设置

Background-image 设置 url 不加 "", 静态检查告警可以忽略。

```
.button2{  
    width: 158px;  
    height: 158px;  
    background-image:url(/common/icon.png);  
}  
  
.button2:active {  
    background-image:url(/common/icon_small.png);  
}
```

2.3 css 样式中设置样式可以通过动态绑定

html:


```
<text class="btn" style="color: {{getColor()}};font-size: {{getSize()}};">Test1</text>
```

js:

```
getColor(){  
    return '#008080'  
},  
getSize(){  
    return '30px'  
},
```

2.4 组件设置背景图

目前容器组件不能设置背景图，有需求可以用 stack 组件，这个组件的特征是后一个子组件会覆盖前一个。

hml:

```
<stack class="stack-parent">  
  <div class="back-child bd-radius"></div>  
  <div class="positioned-child bd-radius"></div>  
  <div class="front-child bd-radius"></div>  
</stack>
```

css:

```
.stack-parent {  
    width: 400px;  
    height: 400px;  
    background-color: #ffffff;  
    border-width: 1px;  
    border-style: solid;  
}  
.back-child {  
    width: 300px;  
    height: 300px;  
    background-color: #3f56ea;  
}  
.front-child {  
    width: 100px;
```

```
height: 100px;
background-color: #00bfc9;
}
.positioned-child {
width: 100px;
height: 100px;
left: 50px;
top: 50px;
background-color: #47cc47;
}
.bd-radius {
border-radius: 16px;
}
```

图2-1 效果图



可以将第一个元素设置为 image，充满屏幕，设置图片，就可以达到背景图的效果。

2.5 JS 文件超过 30KB 报错 “xxx.js is bigger than 30 KB.”

建议精简代码，可以从减少 js 声明方法，减少 js 方法调用，减少外部引入方法来解决 js 文件大小，将配置数据可以使用 Storage 或者 file 存储也可以控制 js 文件大小。

如果报错日志中出现 “file doesn't exit or it's empty, [/user/app/user/ace/run/xxx/assets/js/default/pages/xxx/x.bc]” 日志，则是 IDE 打包时候，没有对应生成 js 文件对应的快照文件，需要重新打包。在 IDE 中先清理编译环境：“Build” → “Clean Project”，然后重新打包：“Build” → “Build Hap(s)/APP(s)” → “Build Hap(s)”。打包过程中需要查看编译日志是否出现 “Failed to convert the xxx.js file to a snapshot.”，如果出现表示对应 js 文件的 bc 文件生成失败，需要对该 js 代码进行精简和配置数据存储化。

2.6 player 设置 streamType

设置 streamType 不支持 player 在播放过程中设置，建议 player 从 init 初始化时设置媒体类型，player 释放 reset 后再次播放也需要重新设置媒体类型。

3

图形常见问题 FAQ

3.1 图形显示问题及建议

3.2 视频显示问题

3.3 内存问题

3.1 图形显示问题及建议

3.1.1 花屏问题

花屏问题常见的原因和排查思路如下：

1. 局部花屏

- 如果出现局部花屏，且与特定控件位置相近，则大概率是控件绘制出现问题。需要排查日志中是否包含 VAU ERROR 和 GRAPHIC ERROR，一旦发现该类型的报错，请联系 GUI 的同事确认问题。

2. 全屏花屏

- 一级滑动过程中出现花屏，松手后即可恢复。这一类问题通常是内部流程异常导致，可以排查日志中异常报错并记录复现路径，联系 GUI 的同事确认问题。
- 界面花屏时，日志中包含内存不足的异常时，通常需要先优化内存类问题。

3. 视频花屏

- 串口输入 "AT^GPU_SAMPLE=screen_cap 1" 保存视频截图，确认是否是视频内容不正确。如果视频截图不正确，请联系媒体的同事确认问题。

- 串口输入 “AT^GPU_SAMPLE=dpu_hide 0”，隐藏图形层，确认是否因为图形层导致的花屏。如果隐藏后，能正常显示，则需要确认 colorkey 值是否配置合理。colorkey 的值应该是图形层中不会存在的像素颜色值。
- 通过 “AT^GPU_SAMPLE=screen_cap 0/1” 分别截取视频层和图形层的内容，并通过 “AT^GPU_SAMPLE=dpu_hide 0/1” 和 “AT^GPU_SAMPLE=dpu_show 0/1” 分别开关视频层和图形层，确认单个图层显示是否符合预期。如果任意一个图层的内容均符合预期，则可能是后端合成出现问题，请联系 GPU 的同事确认问题。

3.1.2 黑屏问题

黑屏问题常见的原因和排查思路如下：

1. 灭屏

- 串口输入 “AT^UIKIT_DFX=dumpstate”，确认 IsScreenOn 的值。如果是 0，则代表是符合预期的灭屏。如果是 1，则需要继续往下排查。

2. Native-JS 切换异常

- 串口输入 “AT^UIKIT_DFX=dumpstate”，确认 NativeRunning 的值。如果是 0，则代表仍处于 JS 应用中，请联系 JS 的同事确认问题。如果是 1，则代表处于 Native 应用中，需要继续往下排查。

3. 控件状态异常

- 串口输入 “AT^UIKIT_DFX=dump_root_view”，确认是否包含有效的控件信息。如果没有，则需要排查业务流程，确保控件被添加到控件树上。如果有符合预期的控件信息，却没有正确地显示，则需要继续往下排查。

4. 后端问题

- 串口输入 “AT^UIKIT_DFX=screencap -f filepath”。该命令会触发实时重绘截图，而并非捞帧拷贝。这意味着，截图数据只能代表当前帧配置的控件内容，但仍然可能因为各种原因与实际送显内容存在差异。
- 串口输入 “AT^GPU_SAMPLE=screen_cap 0”，该命令会保存实际送显的内容。
- 通过 DebugKits 下载文件，并通过 HimgReviser 打开，对比 UIKIT 和 DPU 的截图内容。如果内容一致，均为有效内容，则可能是后端的问题，需要联系 GPU 相关的同事确认。如果内容不一致，DPU 的截图内容是纯黑，而 UIKIT 的截图内容并非纯黑，则需要联系 GUI 的同事确认问题。

3.1.3 冻屏问题

冻屏问题常见的排查方向如下：

1. 如果是静态界面，串口输入“AT^UIKIT_DFX=showtime tp”，滑动屏幕确认是否有 tp 上报的打印，确认是否因为 tp 异常导致的冻屏。
2. 串口输入 3 次“AT^UIKIT_DFX=dumpstate”，确认 GraphicTaskTriggeredCount 是否一直为 2。如果是，则代表被 GraphicEventQueue 中的 Event 阻塞住了，需要排查业务通过 PostGraphicEvent 函数加入队列的任务是否有阻塞的风险。
3. 串口输入 1 次“AT^STACKINFO”，确认是否有媒体线程，且状态一直为 8。如果 2 和 3 均满足，则大概率是视频导致的冻屏，需要联系媒体的同事确认问题。

3.1.4 一级跳转建议

如果从菜单列表或是表盘选择界面选择表盘后跳转至一级界面有明显的延迟，建议适时加载相邻卡片：`void SetHorCurrentPage(uint16_t index, bool loadAdjacent = true)`

如上所示，`UICrossView::SetHorCurrentPage` 函数的第二个参数支持配置是否加载相邻卡片。

建议在一级 `OnStart` 中调用 `SetHorCurrentPage` 时，将 `loadAdjacent` 设置为 `false`，节省切换动画准备过程中加载相邻卡片的耗时。在 `OnResume` 中再次调用 `SetHorCurrentPage`，将 `loadAdjacent` 设置为 `true`。

3.1.5 控件内容未显示

串口输入“AT^UIKIT_DFX=dump_root_view”，根据串口打印的日志确认控件配置：

1. 确认控件是否透明。
2. 确认控件是否可见。
3. 根据父子关系，推导控件是否被其他控件遮挡住。

3.1.6 按钮图标太小，不易点击

按照以下方式，扩大交互区域保持显示效果不变：

1. 扩大控件区域：`UIImageView->Resize/SetPosition`
2. 关闭图片自适应模式：`UIImageView-> SetAutoEnable (false)`
3. 设置图片显示模式：`UIImageView ->SetResizeMode(Center)`

3.1.7 支付宝认证：将无法显示的文字替换为特定的字符

支付宝认证时，给定的字符串中可能包含表情，要求将不可显示的表情替换为特定的字符。建议对文本进行预处理，完成字符替换后，再设置给 UILabel 进行显示：

步骤 1 将 const char* 转 unicode：

```
letter = TypedText::GetUTF8Next(labelLine.text, i, i);
```

步骤 2 确认字符是否存在于字库中：

```
UIFont* fontEngine = UIFont::GetInstance();
GlyphPathData path;
if (!fontEngine->GetGlyphPathData(0x0664, 0, path)) {
    GRAPHIC_LOGE("***** 0x0664 not exist");
}
if (!fontEngine->GetGlyphPathData(0x4E2D, 0, path)) {
    GRAPHIC_LOGE("***** 0x4e2d not exist");
}
```

步骤 3 如果 GetGlyphPathData 返回 true，代表字库中可以查询到当前字符信息；如果返回 false，代表字库中无法查询到当前字符信息，将对应的字符替换为需要显示的字符即可。

----结束

3.1.8 UICollectionViewController 滑动到页面底部时，进度条显示不到 100%

UICollectionView 的进度计算考虑了 scrollBlankSize。如果配置了 scrollBlankSize，且滑到最后的子项时需要显示进度为 100%，可以参照下图，移除 UICollectionView::UpdateScrollBar 函数中“scrollBlankSize_”的处理，将红框的内容替换为绿框中的内容：

```
void UICollectionView::UpdateScrollBar()
{
    Rect allItemsRect = recycle_.GetAdapterItemsRelativeRect();
    // int16_t totalHeight = allItemsRect.GetHeight() + 2 * scrollBlankSize; // 2: two blank spaces on both sides
    int16_t totalHeight = allItemsRect.GetHeight();
    int16_t height = GetHeight();
    yScrollBar_>SetForegroundProportion(static_cast<float>(height) / totalHeight);

    // yScrollBar_>SetScrollProgress(static_cast<float>(scrollBlankSize_ - allItemsRect.GetTop()) /
    //                               (totalHeight - height));
    yScrollBar_>SetScrollProgress(static_cast<float>(- allItemsRect.GetTop()) /
                                  (totalHeight - height));
    RefreshAnimator();
}
```

3.1.9 控件析构时报错：parent is not nullptr

该报错仅用于提示当控件析构时其父控件不为空，并不会导致控件析构失败。如果后续通过父控件获取子节点并进行操作，会访问被释放的指针进而触发异常。建议按照标准流程释放控件，尽可能清除这种报错：先 Remove 后 Delete；先子后父。

3.1.10 UIChartPolyLine 如何合理呈现心率数据

没佩戴手表期间，没有产生心率数据。如果用 0 代表没有数据的时候，并且使用一组数据去显示的话，0 会被判定为谷值，被绘制出对应的谷值点，这并不符合预期。

这种情况下，可以剔除 0，仅使用有效数据，按照时间段分成多组，提前识别包含峰值和谷值的组，对其进行特殊处理：

允许包含峰值的数据组绘制峰值点：EnableTopPoint (true)

允许包含谷值的数据组绘制谷值点：EnableBottomPoint (true)

3.1.11 UIScrollView 使用建议

使用 UIScrollView 时，建议只添加一个 UIViewGroup，起始坐标为[0, 0]，尺寸为内部控件所需的大小。随后再在 UIViewGroup 中添加其他子控件。

常见问题：

判断 UIScrollView 是否滑动到顶部或底部？

当按照上述方式操作后，仅需要判断唯一的子控件的相对坐标即可：

```
if (childrenHead_.GetBottom() <= GetHeight() - 1) {  
    // 已经滑到或超过底部  
}  
if (childrenHead_.GetTop() >= 0) {  
    // 已经滑到或超过顶部  
}
```

3.1.12 事件分发和响应流程

3.1.12.1 表冠事件

同一时期仅有一个控件能获焦，可以通过以下函数设置焦点相关：

- 使控件可获焦: `SetFocusable(bool focusable)`
- 使控件获焦: `RequestFocus()`
- 使控件失焦: `ClearFocus()`
- 设置焦点监听: `SetOnFocusListener(OnFocusListener* onFocusListener)`
- 设置表冠监听: `SetOnRotateListener(OnRotateListener* onRotateListener)`

表冠事件会被分发给当前获焦的控件处理。以下控件包含默认的表冠响应逻辑:

1. `UIPicker`
2. `UIList`
3. `UIScrollView`
4. `UISlider`
5. `UIKaleidoscope`
6. `UIHexagonsList`

3.1.12.2 按键事件

物理按键事件会被分发给 `RootView` 处理, 默认不作任何处理。如需处理, 需要注册按键监听: `void SetOnKeyActListener(OnKeyActListener* onKeyActListener)`

3.1.12.3 触屏类事件

触屏类事件会根据其限定条件, 分发给最上层可响应的控件进行处理:

1. 点击或按下等事件要求控件是可点击的。
设置控件是否可触摸: `SetTouchable(bool touch)`
2. 滑动类事件要求控件是可点击且可滑动的。
设置控件是否可滑动: `SetDraggable(bool draggable)`

如果控件未处理完成, 会将事件传递给其父进行处理, 至到该事件被处理完毕:

- 如果注册了对应的监听, 则以 `Callback` 的返回值判定事件是否已经处理完毕:
`true` 代表事件已经处理完毕; `false` 代表事件仍需传递给父处理。
- 如果没有注册监听, 则以 `"isIntercept_"` 的值为准。可通过 `SetIntercept` 函数配置是否需要拦截事件。

常见问题:

1. 嵌套多级可滑动的控件时, 可能会触发多个控件一起滑动。根据实际需求, 参考以上方式, 适时拦截事件传递。
2. 嵌套不同滑动方向的控件时, 需要根据滑动方向决策当前控件是否处理该事件。

示例：UICrossView 中嵌套了垂直向的 List。垂直向的滑动事件需要给 List 处理，而水平向的滑动事件需要给 UICrossView 处理。事件会被优先分发给 List 处理。因为 List 默认的响应逻辑不会区分滑动事件的方向，所以需要继承 List，重写以下三个函数：

- OnDragStart：获取并记录滑动方向，如果滑动方向与控件响应的滑动方向不符，则返回 false，使事件传递给 UICrossView 处理。
- OnDrag：如果滑动方向与控件响应的滑动方向不符，则返回 false，使事件传递给 UICrossView 处理。
- OnDragEnd：如果滑动方向与控件响应的滑动方向不符，则返回 false，使事件传递给 UICrossView 处理。清除滑动方向标记。

3.1.13 自适应刷新频率

UIKit 会根据当前 UI 任务状态，自动调节刷新频率。当业务空闲时（没有运行中的动画、没有视频、没有待绘制刷新的内容等），刷新频率会自动降为较低的值，从而达到降低功耗的效果。自适应刷新频率对业务代码编写有以下限制：

- Animator：用于生成动画效果，按照最高频率执行动画回调。当动画内容不可见时，需要及时停止动画。不建议通过 Animator 周期性查询数据。
- Task：用户可以控制频率，用于周期性获取数据或处理业务逻辑。任意界面都可按需使用。

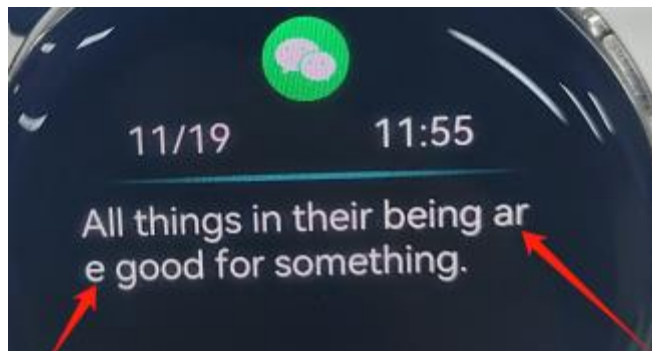
3.1.14 功能宏配置约束

严禁修改以下两个文件中定义的功能宏配置：

middleware\services\gui\uiokit\utils\interfaces\innerkits\graphic_config.h

middleware\services\gui\hal\common\include\graphic_hardware_config.h

3.1.15 多行文字换行时，截断单词



多行文字换行时，出现截断单词的原因是缺少 line_cj.brk 资源，需要将“code\sdk\middleware\services\gui\UIKit\res\”路径下的 line_cj.brk 烧录进板子。

3.2 视频显示问题

3.2.1 视频播放状态异常

3.2.1.1 媒体线程创建失败

媒体线程创建失败，视频没有起播，日志中包含以下错误：

```
[ERR] osThreadNew failed, error type = 0x3000200.  
[00:07:53:663][E]{StartVideoPlay():424} create thread failed
```

错误码 0x3000200 代表创建线程所需要的空间不足，无法创建线程。可采取以下两个手段：

1. 合理分配自己的线程栈大小：在最大业务场景下，串口输入“AT^STACKINFO”，查看自己线程的 Total Size 和 Peak Size，如果 PeakSize 远小于 Total Size，则可以考虑将 Total Size 降低，但在 Peak Size 的基础上仍需预留一定空间。
2. 修改线程部署：联系产品 SDK 接口人协助。

3.2.1.2 内存不足，申请不到视频帧

播放一个视频需要约 1M 内存，图形内存（文字绘制、CanvasExt 绘制、图片资源、截图、视频帧等）严重不足时，可能导致申请视频帧失败。

排查方式：

串口输入“AT^UIKIT_DFX=dumpmem”，确认图形内存使用情况，排查是否有不合理的使用或泄露。

同一时间需确保仅一个视频处于播放状态。如果串口输出的日志中 yuv 大于 2M，意味着当前有多个视频处于播放状态，这会导致内存严重不足，需确认修改视频控制逻辑。

3.2.1.3 视频状态错误

视频状态需要结合业务场景进行适当的处理，尤其需要关注以下几个场景：

- 主页面 CrossView 卡片滑动的生命周期处理：
可以参考“DialVideoView.cpp”中对“video_”的处理。

- 量灭屏视频状态的管理：
注册量灭屏通知，可以参考“DialVideoView.cpp”中 ScreenStatusNotify 函数的处理。
- 弹窗视频状态的管理：
注意时序问题，确保视频停播且删除后，再创建新的视频。

3.3 内存问题

3.3.1 如何快速定位页面内部内存泄漏

图形 GUI 的类都继承了 HeapBase 类，该类自定义了 operator new 和 operator delete 方法，只要继承了 HeapBase 的类的都会被纳入检测中去，在没有打开检测周期时，每 new 一次 HeapBase 的子类，MemCheck 内部维护的 totalCnt_成员都会加一，表示内存中申请对象加一；每 delete 一次 HeapBase 的子类对象，totalCnt_会减一，表示申请内存对象减一。可以通过使用 MemCheck::GetInstance()->DumpMemInfo()打印该变量，如果该变量存在持续增加的情况，可以怀疑这段内存存在内存泄漏，需要打开检测函数进行检测。

使用 void MemCheck::EnableLeakCheck(bool enable)函数划定检测周期，在检测开始地方调用 MemCheck::GetInstance()->EnableLeakCheck(true);，在要检测结束的地方调用 MemCheck::GetInstance()->EnableLeakCheck(false);。在一个检测周期内（例如某段代码，某个对象的构造和析构之间以及 slice 切换之间等），每次申请一个控件对象，内部会记录这个对象的申请次序和大小。当这个周期内同一个申请对象被释放时，这条记录会被移除。当周期结束时，会打印出没有被释放的内存节点信息。这些记录可能没有在这个周期内释放，也可能是内存泄漏，因此需要反复实验，如果发现每次记录的次数和大小都相同，就可以怀疑此处内存泄漏，然后采用 panic 的方式查看调用堆栈。示例如下：

- 步骤 1 为避免细碎内存申请影响定位，可以选择关闭 slice 切换动效（这步操作不必要）。现在关闭 slice 动效，不能直接把 usetransition 改为 false，这会导致 changeslice 生命周期流程异常，后续无法切 slice，如果需要关闭 slice 动画，可以在 KeyInputListener 中设置 slice 变换动效为 TransitionType::TRANSITION_INVALID。

```

void KeyInputListener::OnKeyRelease(const KeyEvent& event)
{
    uint16_t currentViewId = NativeAbility::GetInstance().GetCurSliceId();
    uint16_t keyID = event.GetKeyId();
    if (keyID == static_cast<uint16_t>(ZliteKeyCode::ZLITE_KEY_POWER)) { // 电源按键
        if (longPressFlag == 0 && pressFlag == 1) { // 短按电源键
            if (currentViewId == VIEW_MAEKPHONE_SAMPLE ||
                NotificationManager::GetInstance()->GetWatchCallStatus()) { // 音量卡片
                // 音量+
                CallVolumeDipSwitchControl(VOLUMEUP);
            } else if (currentViewId != VIEW_MAIN_SAMPLE) { // 主表盘
                PressAlarmNotify(); // 当前为闹钟通知页面时，先处理闹钟再切换页面
                NativeAbility::GetInstance().ChangeSlice(VIEW_MAIN_SAMPLE, TransitionType::TRANSITION_ZOOM);
            } else {
                PressAlarmNotify(); // 当前为闹钟通知页面时，先处理闹钟再切换页面
                if (!MainViewSample::GetInstance()->IsMainClockPage()) { // 当前页面为主表盘的其他页面时，切换回主表盘页面
                    MainViewSample::GetInstance()->SwitchToClockPage();
                } else {
                    if (ApplistModel::GetInstance().GetCurrentStyle() == ApplistStyle::WATERFALL_STYLE) {
                        NativeAbility::GetInstance().ChangeSlice(VIEW_APPLIST, TransitionType::TRANSITION_ENTER_WATERFALL);
                    } else {
                        NativeAbility::GetInstance().ChangeSlice(VIEW_APPLIST, TransitionType::TRANSITION_ZOOM);
                    }
                }
            }
        }
    } else if (keyID == static_cast<uint16_t>(ZliteKeyCode::ZLITE_KEY_FUNC)) { // 功能按键

```

步骤 2 在需要检查的 slice 的 presenter 的构造函数中调用 EnableLeakCheck (true)，在析构函数中调用 EnableLeakCheck (false)，并包含头文件。选定检测范围在 presenter 构造和析构之间，如果打印出大量未释放的节点信息，意味着 presenter 中可能存在泄漏。

```

Ability.cpp M    CompassPresenter.cpp M X
nativeapp > nativeui > compass > src > CompassPresen
9   #include "CompassModel.h"
10  #include "graphic_service.h"
11  #include "gfx_utils/mem_check.h"
12

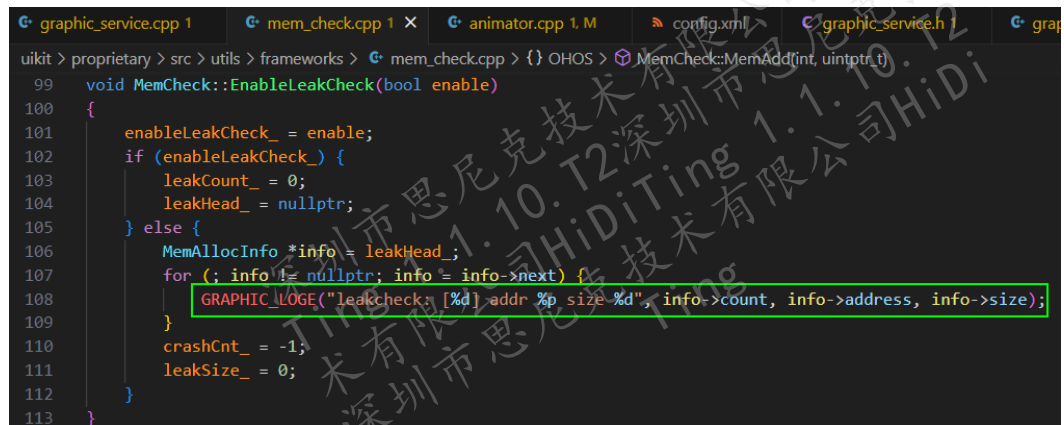
```

```

Ability.cpp M    CompassPresenter.cpp M X    DialBinParser.cpp M
nativeapp > nativeui > compass > src > CompassPresenter.cpp
59  CompassPresenter::CompassPresenter()
60  {
61  |  MemCheck::GetInstance()->EnableLeakCheck(true);
62  |  g_pMainView = this;
63  |  }
64
65  CompassPresenter::~CompassPresenter()
66  {
67  |  MemCheck::GetInstance()->EnableLeakCheck(false);
68  |  g_pMainView = nullptr;
69  |  }

```

步骤 3 烧录版本操作手表进入需要检测的 slice，再退出，查看并记录输出的打印。



步骤 4 多进入几次并记录下打印。反复进入打印出的 count 相同可能是该节点存在泄漏。

```
[11:18:03.429] 收 ← [Graphic Error] [EnableLeakCheck: 108]leakcheck: [262] addr 0x6ca22ae8 size 12
[Graphic Error] [EnableLeakCheck: 108]leakcheck: [271] addr 0x6ca29f64 size 116
[Graphic Error] [EnableLeakCheck: 108]leakcheck: [272] addr 0x6ca3340c size 28
APP [WEARABLE I/] AbilitySliceProxy::CreateSlice
```

步骤 5 输入检测图形内存泄漏的 AT 指令逐个检测内存节点。

```
[Graphic Error] [EnableLeakCheck: 108]leakcheck: [262] addr 0x6ca22ae8 size 12
```

这条 log 的意思是在第 262 次分配内存的地址为 0x6ca22ae8，大小是 12Byte。进入 slice 前，串口输入 “AT^UIKIT_DFX=leak_panic 262 12” 对其进行检测。AT 指令的意思是 “当第 262 次分配内存时，会主动 Panic 崩溃（注意，此时的 12 仅仅是一个校验，当前周期第 262 次申请的内存大小不为 12 时，会给出一个提示）”。此时可以查看 lst 文件，查看 mepc 指针锁定范围。

----结束

3.3.2 如何使用 liteos 定位踩内存

使用 liteos 定位踩内存使用步骤如下：

步骤 1 在

“./kernel/liteos/liteos_v207.1.0/Huawei_LiteOS/kernel/base/mem/bestfit/los_memory_internal.h” 头文件中添加如下代码：

```
#define LOSCFG_MEM_LEAKCHECK 1
#define LOSCFG_BASE_MEM_NODE_INTEGRITY_CHECK 1
#define LOSCFG_MEM_RECORD_LR_CNT 4
#define LOSCFG_MEM_OMIT_LR_CNT 2
```

```
#ifdef LOSCFG_MEM_LEAKCHECK
#define MEM_RECORD_LR_CNT (LOSCFG_MEM_RECORD_LR_CNT - LOSCFG_MEM_OMIT_LR_CNT)
  UINTPTR linkReg[MEM_RECORD_LR_CNT];
#endif
```


需要配置调用栈层数可配置“LOSCFG_MEM_RECORD_LR_CNT”和“LOSCFG_MEM_OMIT_LR_CNT”两个参数，目前调用栈只有两层。

步骤 2 在“./kernel/liteos/liteos_v207.1.0/Huawei_LiteOS/tools/build/config/brandy.config”文件内修改如下，修改后将 output 目录删除再进行编译。

```
201 # LOSCFG_COMPILE_DEBUG is not set
202 LOSCFG_PLATFORM_ADAPT=y
203 LOSCFG_BACKTRACE=y
204 # LOSCFG_ENABLE_MAGICKEY is not set
205 # LOSCFG_DEBUG_VERSION is not set
206 LOSCFG_SERIAL_OUTPUT_ENABLE=y
```

步骤 3 完成步骤 1 和步骤 2 后，编译时需把“-release”编译选项去掉，通过生成的 lst 文件查看调用栈。

步骤 4 如若有踩内存（数组写越界）引起的死机问题，在产生正常的死机文件前会打印相关的内存地址。

----结束

示例：

```
[ERR] [OsMemIntegrityCheck], 1131, memory check error!
memory used but magic num wrong, freeNodeInfo.pstPrev(magic num):0x93d25500
```

```
broken node head: 0x93d25500 0x14 0x6c2daa38 0x80000028,
pre node head: 0x93d255c7 0x14 0x6c2daa10 0x80000024
```

```
broken node head LR info:
```

```
LR[0]:0x705dc8d4
```

```
LR[1]:0x7051feb8
```

被踩的节点

```
pre node head LR info:
```

```
LR[0]:0x7051feb8
```

```
LR[1]:0x7052361e
```

踩该内存地方的节点，
一般情况是前一个
节点踩后一个节点

```
-----
dump mem tmpNode:0x6c2daa5c ~ 0x6c2daa9c
```

```
0x6c2daa5c :93d25500 00000014 6c2daa38 705dc8d4
0x6c2daa6c :7051feb8 80000028 6c2daa50 0000000c
0x6c2daa7c :412001b9 00000000 6c22a9cc 6c22a9cc
0x6c2daa8c :6c2daa5c 00020001 00000000 00000684
```

```
-----
dump mem :0x6c2daa1c ~ tmpNode:0x6c2daa5c
```

```
0x6c2daa1c :705dc8d4 705ea4fa 80000028 6c2da9f8
0x6c2daa2c :00000018 000001b8 6c2daa74 93d255c7
0x6c2daa3c :00000014 6c2daa10 7051feb8 7052361e
0x6c2daa4c :80000024 302e3025 74732066 00737065
```

```
-----
cur node: 0x6c2daa5c
pre node: 0x6c2daa38
pre node was allocated by task:GuiMainThread
APP|exception:b
task:GuiMainThread
thrdPid:0xffffffff
-----
```

Broken node LR0

```
E:\WZQ\GitCode\A03\branch_615\output\brandy\acore\brandy-native-js\...\middleware/services/gui/uikit/proprietary/src/utils/frameworks/mem_custom.cpp:60
if (buf != nullptr) {
    beqz a0, 705dc8a0 < _ZM4OHOS8UIMallocEj+0x1e
E:\WZQ\GitCode\A03\branch_615\output\brandy\acore\brandy-native-js\...\middleware/services/gui/uikit/proprietary/src/utils/frameworks/mem_custom.cpp:61
MemCheck::GetInstance()->MemAdd(s12e, (uintptr_t)buf);
    jal ra, 705dc8d6 < _ZM4OHOS8MemCheckEj11GetInstanceEvp
705dc8c8: d0f410ef    jal ra, 705dc8d6 < _ZM4OHOS8MemCheckEj11GetInstanceEvp
705dc8cc: 85ca        mv a1, s2
705dc8ce: 8626        mv a2, s1
705dc8d0: d5dffeef    jal ra, 705dc8e2 < _ZM4OHOS8MemCheckEj11MemAddEj
E:\WZQ\GitCode\A03\branch_615\output\brandy\acore\brandy-native-js\...\middleware/services/gui/uikit/proprietary/src/utils/frameworks/mem_custom.cpp:63
}
return buf;
705dc8d4: 8526        mv a0, s1
705dc8d6: 99a0        c.poptret (x1,x8,x9,x18),16
```

Broken node LR1

```
E:\WZQ\GitCode\A03\branch_615\output\brandy\acore\brandy-native-js\...\application/wearable/nativeapp/nativeui/main/src/dial/DialLabelView.cpp:58
text = static_cast<char*>(UIMalloc(++textlen));
    addi s3, a0, 1
7051feb2: 854e        mv a0, s3
7051feb4: 203bc0ef    jal ra, 705dc8b6 < _ZM4OHOS8UIMallocEj>
7051feb8: 06a92223    sw a0, 100(s2)
```

pre node LR0

```
E:\WZQ\GitCode\A03\branch_615\output\brandy\acore\brandy-native-js\...\application/wearable/nativeapp/nativeui/main/src/dial/DialLabelView.cpp:58
text = static_cast<char*>(UIMalloc(++textlen));
    addi s3, a0, 1
7051feb2: 854e        mv a0, s3
7051feb4: 203bc0ef    jal ra, 705dc8b6 < _ZM4OHOS8UIMallocEj>
7051feb8: 06a92223    sw a0, 100(s2)
```

pre node LR1


```
E:\W2Q\GitCode\A03\branch_615\output\brandy\acore\brandy-native-js\...\application\wearable\nativeapp\nativeui\main\src\dial\parser\dialbinParser.cpp:425
dialGroup->AddView(static_cast<DialDataType>(data->bindData), view, data->isPeriod);
70523618: 06492583      lw    a0,100(s2)
70523622: 09049583      lh    a1,144(s1)
70523626: 30b4         c.lbu  a3,3(s1)
70523628: 06098613      addi  a2,s3,96
7052362c: ff0fe0ef      jal   ra,70521e1c <ZM40H0513DialViewGroup7AddViewE12DialDataTypePMS 803a1ViewEb>
70523630: a8a1         j      70523688 <ZM40H0513DialbinParser15ParserDialLabelEpb 70 FILE40xex>
```

根据调用栈信息查看相应位置，最终排查到是 DialLabelView.cpp 中出现数组越界和踩内存（这里定义的数组出现越界，应该写法是 `text_[textLen-1] = '\0';`）。

```
uint32_t textLen = static_cast<uint32_t>(strlen(text));
if (textLen > (MAX_BUF_SIZE - 1)) {
    return false;
}
text_ = static_cast<char*>(UIMalloc(++textLen));
if (text_ == nullptr) {
    return false;
}
if (memcpy_s(text_, textLen, text, textLen) != EOK) {
    UIFree(text_);
    text_ = nullptr;
    return false;
}
text_[textLen] = '\0';
return true;
```

3.3.3 运行期间越来越卡

随着运行时间增长，页面操作越来越卡。这种问题通常是由于内存泄露或内存碎片过多，导致申请释放内存越来越慢。

可以结合内存泄露定位手段排查泄露点，优化内存使用。

查看系统内存使用情况 AT 命令：AT+HEAPINFO

查看图形内存使用情况 AT 命令：AT+UIKIT_DFX=dumpmem

3.3.4 一级卡片左右滑动时，卡片内容呈现方形而非圆形

一级卡片滑动动效默认使用方形截图，而非圆形截图。如果要使用圆形截图，需要修改 CardSwipe 对应的派生类的 “isNeedClip_” 成员变量，将其设置为 true。

```

14 class CardSwipe : public HeapBase {
15 public:
16     /*
17      * @brief A destructor used to delete the <b>CardSwipe</b> instance.
18      * @since 1.0
19      * @version 1.0
20      */
21     virtual ~CardSwipe() {}
22
23     /*
24      * @brief Swipe algorithm when drag card.
25      * @param leftCard Left card.
26      * @param rightCard Right card.
27      * @param xOffset Drag distance, xOffset > 0 means left to right, xOffset < 0 means right to left.
28      * @since 1.0
29      * @version 1.0
30      */
31     virtual void CardSwipeAlg(UICardPage* leftCard, UICardPage* rightCard, int16_t xOffset) {}
32
33     /*
34      * @brief Swipe algorithm when drag card.
35      * @param leftCard Left card.
36      * @param rightCard Right card.
37      * @param xOffset Drag distance, xOffset > 0 means left to right, xOffset < 0 means right to left.
38      * @since 1.0
39      * @version 1.0
40      */
41     virtual void CardSwipeAlg(UITableView* leftCard, UITableView* rightCard, int16_t xOffset) {}
42
43     bool isNeedClip = false;
44 };

```

当配置一级滑动动画的 isNeedClip 为 true 时，卡片滑动截图应呈现圆形。如果滑动过程中卡片截图仍呈现方形，日志中会打印内存分配失败，并打印当前内存使用情况。需要结合当前场景，合理优化内存使用。

优化措施：

1. 一级界面卡片滑动时容易出现资源过多，内存不足的问题，需对资源做如下处理：

PreLoad：加载资源，创建控件。

Unload：释放控件，卸载资源。

2. 修改图片压缩配置：

提升图片资源压缩倍率。

可指定不带透明像素的图片输出格式为 RGB888 或 RGB565。

3. 客户界面优化方案，裁剪图片使用：

序列帧资源占用较大时，可以需要抽帧处理。

3.3.5 多线程操作 UI

UIKit 非线程安全，在其他线程中操作 UI 的动作需要通过 PostGraphicEvent 抛到 UI 队列中执行。UI 队列是周期性执行，并不会立即执行。如有必要，客户需要考虑同步机制。

常见问题：

1. 不要在 OsTimer 的 Callback 中直接操作 UI。UI 侧需谨慎使用 OsTimer，建议替换为事件订阅或是 UIKit 中的 Task。
2. 不要在 MsgCenterNotifyProc 中直接操作 UI。
3. 需多加注意因异步执行，时序问题等引起的野指针和空指针问题。可以使用 shared_ptr 以及 weakptr 判断内存是否释放。

Draft

4 BTH 特性使用建议

4.1 对外接口回调函数使用约束

4.2 数传传输场景使用建议

4.3 蓝牙连接相关

4.4 BLE GATT 相关

4.5 协议栈开关

4.6 通话相关使用建议

4.1 对外接口回调函数使用约束

1. 回调函数中的指针内容要拷贝走，不能直接操作回调函数中的指针数据内容。
2. 不能释放回调函数中传入的指针参数。
3. 回调函数实现里建议切换到 APP 任务，避免直接在回调函数中执行复杂流程导致 BTH 栈溢出；在回调处理中也不能有 sleep 等耗时的操作。
4. 不能在回调函数中直接调用 BTH 提供的对外接口，切换任务后由 APP 任务调用。

4.2 数传传输场景使用建议

1. 使用 BR SPP 或者 BLE GATT 写命令或者通知功能实现数据传输协议时，建议上层应用实现数传校验功能。例如一次传输 n 包后等待接收端校验，校验通过后再继续下一轮传输，否则根据校验结果进行数据重传。

2. OTA 升级传输过程中建议停止蓝牙相关的其他业务，例如微信消息同步、表盘传输、音乐播放、通话接听等。
3. 通话过程中建议不进行大数据的传输，例如表盘传输、微信消息推送等。
4. 音乐场景下建议不进行大数据的传输，例如表盘传输、微信消息推送等。
5. 建议 SPP 数据传输和 BLE GATT 数据传输不同时进行。
6. 协议栈默认流控限制字节数为 51200Byte，如果数传压测时出现 B 核内存不足重启现象，可以调用 `gap_bt_set_flow_limit` 接口适当调低该流控限制字节数，值范围是 3072 ~ 51200 Byte，建议按照 1024Byte 递减尝试验证，直到不出现 B 核内存不足重启现象。蓝牙音乐流控限制可调节最小值为 10240Byte。

4.3 蓝牙连接相关

1. BT 回连策略建议 APP 层根据具体需求自己实现，默认提供的回连策略不一定能满足特定场景会连接效率、连接功耗的要求。
2. BLE 的回连策略由上层应用实现，断连后自动打开广播待手机连接。
3. BT 连接/回连过程中不建议同步做 BLE 空口业务，例如服务发现、GATT 数据读写通知等。
4. BLE 配对过程中不建议做大批量数据传输和 sink 音乐播放。
5. 一键双连仅限双模设备与苹果系统 ios13 以上使用的功能，与安卓手机连接，建议关闭一键双连功能。
6. 以下场景，蓝牙协议栈会打开 BR 广播，上层应用使用关闭广播方式实现关闭蓝牙时，需要在以下逻辑之后再关闭广播：

在回调函数 `typedef void (*gap_scan_mode_changed_callback)(int mode)` 中，判断是关闭蓝牙场景且上报的是打开广播且是 ACL 断连时，再关闭广播。

- a. 断连之后，如果有配对信息，会把 BR 广播模式设置成可连接；如果没有配对信息，会把 BR 广播模式设置成可扫描可连接。
 - b. 配置蓝牙地址接口，会把 BR 广播模式设置成可扫描可连接。
7. 对于双模设备，配置蓝牙地址和配置蓝牙名称的时机，建议在 `enable_bt_stack` 之后、`gatts_register_server` 之前执行。

4.4 BLE GATT 相关

1. 添加服务不宜太多，特征不能太大，所有服务占用的内存空间建议不超过 2KB。
2. 为了提升数据传输效率，连接后建议先更新 MTU 值到最大（517Byte），具体更新方法如下：

GATT MTU 交换需要 client 端发起，如果板侧作 client 端，请调用 `gattc_exchange_mtu_req` 接口发起 MTU 交换请求。如果板侧作 server，只能等待端 client 发起 MTU 交换请求，协商结果通过回调函数 `gatts_mtu_changed_callback` 上报。

📖 说明

MTU 值是两端设备协商的结果，对于一些低版本的蓝牙设备，可能不支持 517Byte 的 MTU，需根据实际情况选择合适的值进行更新。

3. 服务异步添加接口：

`gatts_add_service`、`gatts_add_characteristic`、`gatts_add_descriptor` 均为异步添加接口，对应的添加完成回调表示添加成功。

📖 说明

添加 characteristic 应当在对应的 service 添加完成回调中执行。

添加 descriptor 也应当在对应的 characteristic 添加完成回调中执行。

添加下一个服务时，要在上一个服务 start 回调成功后执行。

添加服务较多时，建议用服务同步添加接口。

4. 服务同步添加接口：

`gatts_add_service_sync`、`gatts_add_characteristic_sync`、`gatts_add_descriptor_sync` 均为同步添加接口。函数返回后即表示添加成功。

5. 服务端数据传输反压：

`gatts_notify_indicate`、`gatts_notify_indicate_by_uuid` 接口在返回值 `ERRCODE_BT_BUSY` 时，代表 B 核数传繁忙，本次传输失败。建议此时等待并重试。

6. 客户端服务发现：

`gattc_discovery_service`、`gattc_discovery_character`、`gattc_discovery_descriptor` 接口依赖空口响应，建议使用时指定对应的 UUID。

7. 客户端数据传输反压：

`gattc_write_req`、`gattc_write_cmd` 接口在返回值为 `ERRCODE_BT_BUSY` 时，代表 B 核数传繁忙，本次传输失败。建议此时等待并重试。

4.5 协议栈开关

1. enable_ble、disable_ble 接口只在 BLE ONLY 的版本使用。
2. 在关闭蓝牙协议栈之前，建议先主动停止当前数据传输业务，主动注销应用层注册的 GATT client，然后再关闭蓝牙协议栈。
3. 在一些低功耗业务场景下，只需要关闭广播和断开蓝牙连接即可，不调用 disable_bt_stack 关闭蓝牙协议栈。
4. a2dp snk 服务开关，请参照以下内容：
 - a. 打开 a2dp snk 服务，请先调用 **a2dp_snk_service_register** 接口注册 a2dp snk 服务，接着再根据场景判断是否调用 **a2dp_snk_connect** 接口发起 a2dp snk 服务连接。
 - b. 关闭 a2dp snk 服务，请先判断 a2dp snk 是否处于已连接状态，如果是，请先调用 **a2dp_snk_disconnect** 接口断开 a2dp snk 连接，再调用 **a2dp_snk_service_unregister** 接口。

4.6 通话相关使用建议

1. SCO 的创建与销毁
 - 收到 HFP_SCO_STATE_CONNECTING 回调时，调用媒体接口创建 SCO 通路。
 - 收到 HFP_SCO_STATE_DISCONNECTED 回调时，调用媒体接口销毁 SCO 通路。
 - 其他流程不可以主动调用媒体接口创建或者销毁通路。
 - 可以调用 hfp_hf_disconnect_sco 接口主动断开 SCO，收到回调后销毁 SCO 通路。
2. 通话状态
 - 收到 HFP_HF_CALL_STATE_INCOMING 回调时，手表可以显示来电中。
 - 收到 HFP_HF_CALL_STATE_INCOMING 回调时，有 SCO 连接通话在手表端，手表可以显示通话中；无 sco 连接通话在手机端，手表端可以不显示通话中。
 - 收到 HFP_HF_CALL_STATE_FINISHED 回调时时，手表退出通话中显示。
 - Hfp 断连时，手表要退出来电中、通话中等显示。

3. 微信通话

- 正常手表拨打电话会经过 HFP_HF_CALL_STATE_ALERTING 状态，再进入 HFP_HF_CALL_STATE_ACTIVE 状态。
- 正常手表接听电话会经过 HFP_HF_CALL_STATE_INCOMING 状态，再进入 HFP_HF_CALL_STATE_ACTIVE 状态。
- 微信通话的简易识别方法：微信通话时，不会经过 HFP_HF_CALL_STATE_ALERTING 状态或者 HFP_HF_CALL_STATE_INCOMING 状态，而是直接进入 HFP_HF_CALL_STATE_ACTIVE 状态。但是对于蜂窝通话进行中才开始连接手表的场景会误识别为微信通话的情况。

4. 带内带外铃声

- 带外铃声本地响铃实现方案：

实现 hfp_hf_in_band_ring_tone_changed_callback 回调函数并注册回调函数，实现举例如下：

```
static void sample_handle_inband_ringtones_changed(const bd_addr_t *bd_addr, int32_t status)
{
    if ((status & HFP_HF_IN_BAND_TONG) == 0) { /* 带外铃声 */
        if ((status & HFP_HF_INCOMING_CALL) != 0) { /* 带外铃声来电振铃 */
            /* 调用媒体接口本地媒体响AUDIO_STREAM_RING */
        } else if ((status & HFP_HF_TERMINATE_LOCAL_RINGTONE) != 0) { /* 终止本地生成的振铃音 */
            /* 调用媒体接口本地媒体结束响铃AUDIO_STREAM_RING */
        }
    }
}
```

- 带内铃声本地响铃实现方案

实现 hfp_hf_in_band_ring_tone_changed_callback 回调函数并注册回调函数，实现举例如下：

```
static void sample_handle_inband_ringtones_changed(const bd_addr_t *bd_addr, int32_t status)
{
    if ((status & HFP_HF_IN_BAND_TONG) != 0) { /* 带内铃声 */
        if ((status & HFP_HF_INCOMING_CALL) != 0) { /* 带内铃声来电振铃 */
        }
    }
}
```



```
/* 调用媒体接口静音通话带内铃声  
AUDIO_STREAM_VOICE_CALL_BT_SCO, 本地媒体响铃AUDIO_STREAM_RING */  
  
} else if ((status & HFP_HF_TERMINATE_LOCAL_RINGTONE) != 0) { /* 终止  
本地生成的振铃音 */  
  
/* 调用媒体接口取消静音通话带内铃声  
AUDIO_STREAM_VOICE_CALL_BT_SCO, 本地媒体结束响铃AUDIO_STREAM_RING  
*/  
    }  
}  
}
```

5 音频常见问题 FAQ

- 5.1 音频问题定位方法
- 5.2 无声问题
- 5.3 卡顿问题
- 5.4 音量问题

5.1 音频问题定位方法

表5-1 各场景的音频通路及最低主频要求

场景	音频通路（加粗部分为 DSP 中的音频通路）	主频
蓝牙音乐	Acore→ ADP→ADEC→TRACK→SOUND→AENC→ADP →BTcore	64M
本地音乐	Acore→ ADP→ADEC→TRACK→SOUND →SmartPA	147M
录音	MIC→ AI→SEA→AENC→ADP →Acore	147M
CAT1 通话	上行：MIC→ AI→SEA→TRACK→SOUND →CAT1 下行：CAT1→ AI→TRACK→SOUND →SmartPA	176M
蓝牙通话	上行：MIC→ AI→SEA→AENC→ADP →BTcore 下行：BTcore→ ADP→ADEC→TRACK→SOUND →SmartPA	176M
命令词识别	MIC→ AI→SEA	294M

📖 说明

以下场景的音频通路一致，在分析音频问题时可视为同一场景：

1. 本地音乐 = 视频播放 = 微信语音
2. 录音 = 微信录音

步骤 1 通过 proc 命令查询 AB 模块信息，检查音频通路是否创建、DSP 主频配置是否正确（请参见《HiDiTingV100 音频驱动 调试指南》“AB 模块”章节）。

步骤 2 沿音频通路逐级通过 proc 命令查询数据量是否有过载、欠载现象。

步骤 3 沿音频通路逐级通过 dump 工具抓取输出数据分析。

----结束

5.2 无声问题

根据错误码分类，常见无声问题的可能原因及解决方法如下：

📖 说明

若需查询音频报错，请在串口日志中搜索关键字：[A][ERROR]

- **串口有 0x8000502D 报错，后续创建通路时概率出现创建失败且有 0x80005037 报错：**
销毁通路时有时序问题。请咨询媒体和蓝牙同事，保证接口调用时序。
- **串口无音频报错：**
 - a. 使用了错误的音频输出端口。请检查
“drivers\boards\brandy_evb\product\product_evb4_standard.h” 中的音频输入端口宏 BUILTIN_AI_PORT 和音频输出端口宏 BUILTIN_SND_OUT_PORT 是否与硬件接线一致。
 - b. SmartPA 芯片驱动未正确适配，导致 SmartPA 芯片未启动。请参考《HiDiTingV100 软件开发指南》“音频相关特性”章节正确适配 SmartPA 芯片驱动。
 - c. 错误地在音乐播放后调用了 SmartPA 芯片驱动的 stop 或 deinit 接口。请参考《HiDiTingV100 软件开发指南》“音频相关特性”章节，把需要关闭 SmartPA 芯片的函数实现注册到 deinit 接口中即可，不需要额外调用。

5.3 卡顿问题

根据错误码分类，常见卡顿问题的可能原因及解决方法如下：

📖 说明

若需查询音频报错，请在串口日志中搜索关键字：[A][ERROR]

- **串口无音频报错：**
 - a. 检查前级送入的数据是否卡顿。
 - b. 通过 proc 命令查询 AB 模块信息，检查 DSP 主频是否设置正确，请参见《HiDiTingV100 音频驱动 调试指南》“AB 模块”章节。

5.4 音量问题

- 上行音量问题（通话、录音）

上行目前未向上层暴露音量设置接口。
- 下行音量问题（通话、音乐、提示音）

先将音量值调到最大，然后沿音频通路逐级通过 dump 工具抓取输出数据分析。（请参见《HiDiTingV100 音频驱动 调试指南》“音频 DUMP 工具”章节）

 - a. 若经过 track 模块后，音量降低：

请检查 proc 信息（请参见《HiDiTingV100 音频驱动 调试指南》“音频 PROC 工具-AO 模块”章节）中的 volume_integer 是否为 0。
 - b. 若经过 sound 模块后，音量仍未降低：

请联系 PA 厂商帮忙调试。